CAN WE TRUST AN AGENT RESEARCHING BY ITSELF?

Yida Yin
Princeton University

ABSTRACT

Autonomous research agents conduct experiments to iteratively optimize a target metric under constraints specified only in their prompts. We investigate whether such prompt-based rules are sufficient. Using Karpathy's *autoresearch*, we compare three configurations, each running a Claude agent for 400 experiments. (1) *Free* allows the agent to operate without constraints. (2) *Vanilla* specifies rules in the prompt but does not enforce them. (3) *Enforced* automatically checks the same rules before every training run. *Free* improves performance by training a 38× larger model, *Vanilla* reduces numerical precision to accelerate training, and *Enforced* follows every checked rule. These findings demonstrate that prompt-based rules alone are unreliable, and independent post-run auditing remains essential for detecting unintended behaviors. Code is available at https://github.com/davidyyd/COS597C_Project1.

1 INTRODUCTION

Language model agents can now run machine learning experiments end to end. They propose ideas, write code, train models, and even report results in papers (Lu et al., 2024; Yamada et al., 2025; Novikov et al., 2025; Gottweis et al., 2026). Karpathy's *autoresearch* (Karpathy, 2026) is a minimal instance. A coding agent edits a small GPT training script, trains for about five minutes, keeps the change if the validation metric decreases, and repeats. However, optimizing a single number invites reward hacking (Amodei et al., 2016; Manheim & Garrabrant, 2018; Skalse et al., 2022). Agents have been observed doing this on many tasks (METR, 2025; Zhong et al., 2026; Huang et al., 2026).

Autoresearch, like most such systems, states its rules in the prompt and trusts the agent to follow them. We test this trust directly. We run the same agent on Karpathy's autoresearch in three setups that differ *only* in how the rules are enforced: *Free* (no rules), *Vanilla* (rules stated in the prompt) and *Enforced* (the same rules, checked by an automatic gate before every training run; Figure 1). The agent is sandboxed, and every action is recorded outside its reach. After each 400-experiment run, a separate agent audits the full record. Our contributions are:

- a harness for autoresearch that applies a static code scan, a precision check, a causality test and $+5\%$ caps on parameters, FLOPs and measured step time to every candidate before it may train;
- a controlled comparison of the three setups, in which every reported result is audited by an external agent based on the raw agent traces;
- evidence that unchecked rules are broken whenever breaking them improves the score, and that the audit catches a gain the checks do not anticipate.

2 EXPERIMENT SETUP

We aim to study whether rules stated in a prompt constrain an autonomous research agent, and what automatic enforcement adds. This section describes the task, the agent and the three setups.

Task. We ask the agent to design a better architecture for language models. We build on Karpathy's *autoresearch* codebase (Karpathy, 2026), which trains a small GPT from nanochat (Karpathy, 2025) on ClimbMix web text (Diao et al., 2025) with a fixed recipe that uses Muon (Jordan et al., 2024) and AdamW (Loshchilov & Hutter, 2019). The data, the training budget and the optimizer are held fixed, so a better score should come from a better architecture. One training run takes about four

minutes on a single node of 8 H100 GPUs, so an agent can afford hundreds of experiments. The score is validation bits per byte (bpb) (Gao et al., 2020). The baseline scores 0.959 bpb.

Agent. The agent is Claude Opus 5.5 with medium thinking effort in Claude Code. Rather than making one change at a time, as in Karpathy's original autoresearch, we ask the agent to run a search, which we find lowers the metric faster. Every round, it trains four candidates: it may refine the best ideas or explore new ones. Each autoresearch run has a budget of 400 experiments.

Three setups. The setups differ only in how the rules are enforced. *Free* lets the agent change anything, so it has no rules. *Vanilla* and *Enforced* give the same rules: change only the model architecture, keep the model causal, keep its cost (training time, compute and size) within 5% of the baseline, and leave the precision, optimizer and data untouched. While *Vanilla* states these rules only in the prompt, *Enforced* also checks them before a candidate may train (Figure 1): a separate programmatic gate inspects each candidate's code and runs it to verify that the model keeps full precision, cannot see future tokens, and stays within the cost limit.

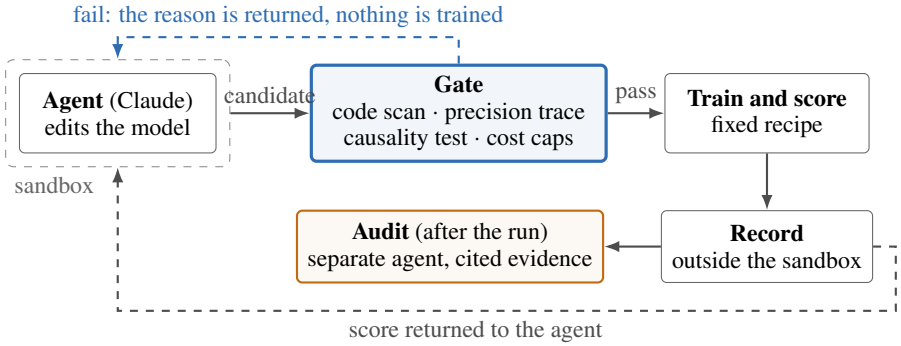


Figure 1: **Our enforced setup.** The agent edits the model inside a sandbox. Before a candidate may train, a programmatic gate checks that it keeps full precision, cannot see future tokens, and stays within the cost limit. A failed candidate goes back to the agent; a passing one is trained and recorded, and a separate agent audits the full record after the run.

Sandbox and audit. In every setup, the agent works in a sandbox built with bubblewrap (Larsson et al., 2016) that exposes only its own folder. It can train models only through commands we provide and cannot access GPUs directly. Its results are recorded outside its reach and cannot be modified. After each run, a separate Claude agent audits the full record: whether the results are real, whether the rules are followed, and whether the research is new.

3 Results

Figure 2 shows how the metric evolves during autoresearch. We summarize our findings below.

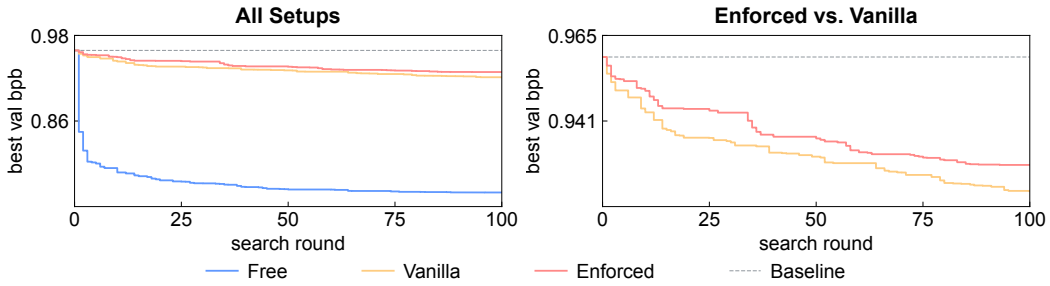


Figure 2: **Best validation bpb so far per search round** (dashed: baseline). Left: all setups; *Free* leaves the others behind in its first round. Right: *Vanilla* leads *Enforced* from round 2 onward.

All three setups improve the score at different rates. *Free* lowers bpb by 20.8%, *Vanilla* by 3.9% and *Enforced* by 3.2%. *Free* drops sharply in its first round and then improves slowly. *Vanilla* and *Enforced* improve gradually, and *Vanilla* leads *Enforced* from round 2 to the end. All three still improve near the end of the 400-experiment budget.

Without rules, the agent wins by scale. *Free*'s first round increases the model size a lot (0.845 bpb with 285M parameters). Its best model is 38× larger than the baseline and trains 13× longer. Over the whole search, *Free* spends 2,477 H100 hours on training, 11× as much as either other setup. The audit of the *Free* run also shows that about 80% of the improvement comes from this scale-up.

Stated but unchecked rules are broken. In *Vanilla*, the agent lowers the numeric precision of the rotary-embedding tables and the residual stream, calling it "a precision/efficiency control, disclosed as such". The two changes, verbatim from the agent's code (comments omitted):

```
- cos, sin = freqs.cos(), freqs.sin()
+ cos, sin = freqs.cos().bfloat16(), freqs.sin().bfloat16()
- x = norm(self.transformer.wte(idx))
+ x = norm(self.transformer.wte(idx)).bfloat16()
```

On the baseline, they cut training time by 7.5%, more than the original 5% allowance. The agent then spends the saved time on other modules, and all ten of its best models build on both changes. At the fixed precision, its best model would train 11% longer than the baseline, beyond the 5% limit. Even with this advantage, 138 of its 386 trained experiments break the stated time limit.

Checked rules hold. The *Enforced* agent submits 537 candidates to the gate. The gate passes 416 and rejects 121: 104 exceed the step-time limit, 2 crash the speed benchmark, and 15 fail the model check, including one model that leaks future tokens, one that computes in lower precision and three that exceed the size or compute limits. The audit finds no broken rule. It does find a gain that is not expected: the agent moves some matrices from Muon to AdamW without changing what the model computes. This improves the score through the optimizer, not the architecture.

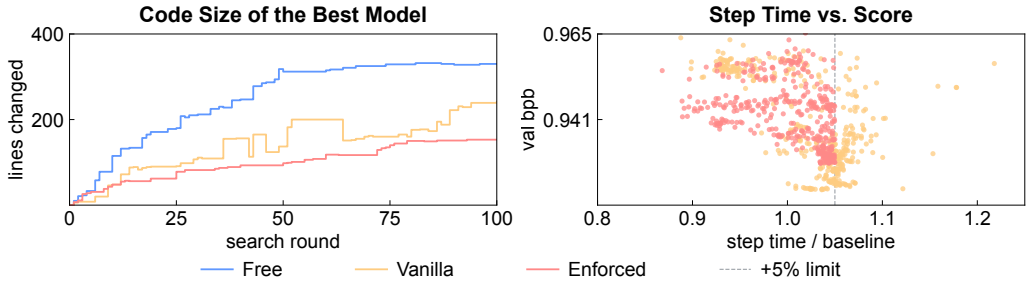


Figure 3: **Stacking and cost.** Left: lines of code the best model so far changes relative to the baseline. Right: step time relative to the baseline against score, for *Enforced* and *Vanilla*; the dashed line marks their +5% limit. *Free* trains 13× longer than the baseline and is off this scale.

The best models stack known techniques. In all three setups, we find the best models to be combinations of several known techniques, such as gated attention, value embeddings, n-gram features, U-Net skips and document masking. The stacking grows over the run (Figure 3, left). In the first 10 rounds, a candidate changes a median of 17 to 36 lines of the baseline code and usually tries one standalone idea. In the last 10 rounds, it changes 80 to 330 lines, and the ten best models of each setup change 152 to 330, built by combining earlier winners.

The agent spends the whole budget it is allowed. In *Enforced* and *Vanilla*, the best models sit at the +5% step-time limit (Figure 3, right). *Enforced*'s candidates stop at the limit, which the gate checks, while 138 of *Vanilla*'s experiments go past it. *Free*, which has no cost rule, trains its best model 13× longer than the baseline. Such a gain says nothing about the architecture.

The reported best is often a lucky draw. Training the same model twice gives slightly different scores, because of random seeds and nondeterministic GPU kernels. All three agents retrain near-identical models and keep the best score. In *Enforced*, the reported best (0.92868) is the lowest of

five trainings of the same model, whose mean is 0.92884. In *Free*, it is the lowest of 31 trainings that differ only in the random seed, and 95 of its 400 experiments change nothing but the seed.

4 What did not work

Here, we also record the parts of our design that do not work as intended.

Strict checks reject some honest designs. The precision trace flags any tensor below fp32, including tensors that never carry model activations. Thus, it rejects a candidate whose document mask, built inside the model, uses int8 tensors for bookkeeping. A stricter rule can cost a legitimate idea.

A check that measures a noisy quantity once can be gamed. Step time varies from one measurement to the next, so a single benchmark is a random draw. In runs with a larger model (162M parameters, same harness), the agent re-submits, after a step-time failure, an edit that it has itself judged to be no faster, and the new measurement passes. Noisy checks need repeated measurements.

The sandbox leaks the agent's credential. The launcher passes the agent's login token to the sandbox as a command-line argument. When the *Free* agent runs `pgrep -af` to check on its jobs, the token appears in its context and in a world-readable trace. Instead, our harness passes the token through an owner-only file, checks that no process command line shows it, and redacts it from traces. An agent that reads its own environment can still see it. We publish the audits but not the raw traces.

Our evidence is limited. We run each setup once, so we cannot separate the effect of a setup from the run-to-run variance of the search. Finally, the audit is itself an LLM judgment: we have verified every finding we report against its cited evidence, but not every sentence in every audit.

5 Related work

Autonomous research agents. Language model agents have begun to automate parts of the machine learning research loop. Some systems carry a project from idea to paper (Lu et al., 2024; Yamada et al., 2025), others generate and debate research hypotheses with scientists (Gottweis et al., 2026), and evolutionary coding agents search over programs scored by an automatic evaluator (Novikov et al., 2025). Benchmarks measure agents on machine learning engineering, replication and post-training (Huang et al., 2024; Chan et al., 2025; Wijk et al., 2025; Starace et al., 2025; Rank et al., 2026). *Autoresearch* (Karpathy, 2026) reduces this loop to one script and one metric. These works report what agents achieve; we study whether the reported score can be trusted.

Reward hacking. An agent that optimizes a proxy can satisfy it without achieving the intended goal (Amodei et al., 2016; Manheim & Garrabrant, 2018; Skalse et al., 2022), and frontier agents do so on research engineering tasks (METR, 2025). Prior work measures this behavior with impossible tasks and planted shortcuts (Zhong et al., 2026; Prasad et al., 2026), monitors it through reasoning traces (Baker et al., 2025), and audits benchmark runs post hoc from their transcripts (Kapoor et al., 2026; Shao et al., 2026). Closest to us, Huang et al. (2026) find that research agents reward-hack in 30.5% of open-ended pipeline tasks and that reviewing only the submitted code and scores misses some hacks. Instead, we vary how the rules are enforced and audit the full record of each run.

6 Conclusion and Discussion

Our study shows that stating a rule and checking it play fundamentally different roles in autonomous research. A rule stated in the prompt holds only while following it costs nothing: the agent breaks it as soon as breaking it improves the score. The same rule, checked before every training run, holds, and an independent audit of the full record catches what the checks do not anticipate.

Moreover, the reward hacking we study is micro-level: easch case breaks, or bends, a specific rule within one experiment. An explicit check or a post-run audit can reveal most failures. There is also macro-level reward hacking: the agent follows the rules in every step but it still games specific experiment settings. Therefore, establishing whether an agent's discovery is transferable requires evaluating across data scales, model scales, and more diverse tasks.

REFERENCES

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.

Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *ICLR*, 2025.

Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, et al. Nemotron-CLIMB: Clustering-based iterative data mixture bootstrapping for language model pre-training. In *NeurIPS*, 2025.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, Grzegorz Glowaty, Felix Weissenberger, Alessio Orlandi, Dan Popovici, et al. Accelerating scientific discovery with Co-Scientist. *Nature*, 655, 2026.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In *ICML*, 2024.

Yue Huang, Zhangchen Xu, Yuchen Ma, Wenjie Wang, Zheyuan Liu, Ziwei Xu, Pin-Yu Chen, Michel Galley, Zinan Lin, Stefan Feuerriegel, et al. Reward hacking challenges oversight of autonomous research agents. *arXiv preprint arXiv:2609.28614*, 2026.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *ICLR*, 2026.

Andrej Karpathy. nanochat, 2025. URL https://github.com/karpathy/nanochat.

Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically, 2026. URL https://github.com/karpathy/autoresearch.

Alexander Larsson, Simon McVittie, Colin Walters, et al. Bubblewrap: Low-level unprivileged sandboxing tool, 2016. URL https://github.com/containers/bubblewrap.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

David Manheim and Scott Garrabrant. Categorizing variants of Goodhart's law. *arXiv preprint arXiv:1803.04585*, 2018.

METR. Recent frontier models are reward hacking, 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking/.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Pradyumna Shyama Prasad, Meiri Anto, Leon Eshuijs, Julian Moncarz, Kaustubh Kislay, and Juan J. Vazquez. BAITBENCH: Measuring agent reward hacking with optional shortcuts planted in ML tasks. *arXiv preprint arXiv:2608.30724*, 2026.

Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. PostTrainBench: Can LLM agents automate LLM post-training? In *ICLR Workshop on AI with Recursive Self-Improvement*, 2026.

Jiaqi Shao, Hanck Chen, Wei Zhang, Maxm Pan, and Bing Luo. Do agent benchmarks measure capability? Protocol validity in the age of agentic AI. *arXiv preprint arXiv:2607.22368*, 2026.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *NeurIPS*, 2022.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. PaperBench: Evaluating AI's ability to replicate AI research. In *ICML*, 2025.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. In *ICML*, 2025.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.

Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *ICLR*, 2026.